Technify Academic AI Assistant

(TAIA)

AI Team 1 - Project Document

Technify Software House

Duration: June 7, 2026 - July 31, 2026 (8 Weeks)

Team Size: 6 Members

GitHub: https://github.com/zotac-pc/technify-ai-assistant

CONFIDENTIAL - For AI Team 1 Members Only

Document Version 1.0 | June 7, 2026

Table of Contents

1. Project Overview

Technify is building a University ERP (Enterprise Resource Planning) system - a complete university management software. Multiple intern teams are working on different parts:

Team	Technology	Responsibility
Web App	React + Next.js	ERP web frontend
Backend	Laravel + PostgreSQL	ERP backend & database
Mobile	Flutter	ERP mobile app
AI Team (Us)	Python + FastAPI	AI-powered academic assistant
Data Science	Python	Analytics & dashboards

OUR SCOPE: We build ONLY the AI Assistant microservice. We do NOT build the ERP itself. Our AI service is a separate application that communicates with the ERP through REST APIs.

The Golden Rule

The AI Assistant is NOT allowed to access the database directly. All data access must occur through secure ERP APIs.

2. What We Are Building

We are building a smart AI chatbot that sits inside the university ERP system. When a user (student, teacher, or admin) opens the ERP, they can talk to the AI assistant in natural language.

Example Conversations

```
Student: "What is my attendance in Web Engineering?"
AI: "Your attendance in Web Engineering (CS-301) is 78%.
     You have attended 25 out of 32 classes."

Faculty: "Which students have low attendance?"
AI: "8 students have attendance below 75% in Database Systems..."

Admin: "What is the fee collection status?"
AI: "Total Expected: PKR 425M | Collected: PKR 382.5M (90%)"
```

3. How It Works - System Flow

- Step 1: User logs into ERP (web or mobile app)
- Step 2: ERP authenticates the user and creates a JWT token
- Step 3: User opens the AI Assistant (chatbot widget in the ERP)
- Step 4: ERP sends UserID + Role + JWT Token to our AI service
- Step 5: Our AI validates the JWT token (checks signature & expiry)
- Step 6: AI understands the question using LLM (GPT or Llama)
- Step 7: AI calls the appropriate ERP API to fetch data
- Example: GET /api/v1/student/STU-0042/attendance
- Step 8: ERP returns ONLY the data this user is allowed to see
- Step 9: AI formats a human-readable response using the LLM
- Step 10: User sees the answer in the chat window

4. System Architecture

Our system follows a microservice architecture. The AI assistant is completely separate from the ERP. The ERP team builds the frontend (React), backend (Laravel), and database (PostgreSQL). We build the AI service (FastAPI + LangChain + ChromaDB).

Key components of our AI microservice:

Component	Purpose
FastAPI Gateway	Receives requests, validates JWT, routes to AI engine
LangChain Engine	Processes questions, determines intent, calls tools
ERP Connector	HTTP client that calls ERP backend APIs
ChromaDB (RAG)	Vector database storing university policy documents
LLM (GPT/Llama)	The AI brain that generates natural language responses
Audit Logger	Logs every request for security compliance

Why separate microservice? (1) Independent development - we use Python, ERP uses PHP. (2) Independent deployment. (3) Reusability in other Technify products. (4) Independent scaling.

5. Technology Stack

Technology	Version	Purpose
Python	3.10+	Core programming language
FastAPI	0.115+	Web framework - our API server
LangChain	0.3+	AI/LLM framework - prompts, RAG, tools
ChromaDB	1.0+	Vector database for document search
OpenAI GPT	Phase 1	LLM brain (switch to Llama in Phase 2)
JWT	3.4+	Authentication token validation
httpx	0.28+	HTTP client for calling ERP APIs
Faker	37+	Synthetic test data generation
pytest	8.0+	Automated testing framework

6. The 5 Modules We Must Build

Module 1: User Authentication

Validate JWT tokens from ERP, check user role and permissions before answering any question. Reject expired or fake tokens.

Module 2: Academic Info Retrieval

Call ERP APIs to fetch attendance, results, GPA, timetable, assignments, fee records. Format data for the LLM.

Module 3: Knowledge Base (RAG)

Store university policy documents in ChromaDB. When users ask policy questions, retrieve relevant document chunks and generate accurate answers.

Module 4: Study Recommendations

Generate personalized study schedules, recommend courses, suggest learning resources based on student's academic data.

Module 5: Conversation Management

Maintain context across multiple messages. Follow-up questions like 'What about Web Engineering?' should work correctly.

7. Features By User Role

Student Features

- Check attendance (overall and per course)
- View pending assignments
- Check exam schedule
- View GPA and results
- Check fee status
- View registered courses and timetable
- Generate study plans
- Ask about university policies

Faculty Features

- View students with low attendance
- Check ungraded assignments
- Identify at-risk students
- View course performance statistics

Admin Features

- View total enrollment numbers
- Admission statistics
- Fee collection reports
- Department-wise performance

8. Security & Privacy

RBAC (Role-Based Access Control): Every request is checked against the user's role. Students see only their own data. Faculty see only their assigned courses. Admins see aggregated statistics.

The AI Must NEVER:

- Modify marks, attendance, or fees (read-only access)
- Approve admissions
- Access unauthorized data
- Reveal another student's information (0% data leakage)

Audit Logging

Every request is logged with: UserID, Role, Query, Response Type, Timestamp, Response Time. Logs are stored for compliance and security review.

9. Project Folder Structure

```
technify-ai-assistant/
|-- app/                    # Main application code
|   |-- main.py             # FastAPI entry point
|   |-- auth/               # Module 1: JWT & RBAC
|   |-- api/routes/         # API endpoints
|   |-- services/           # Business logic (AI, ERP connector, KB)
|   |-- chains/             # LangChain chains (student, faculty, admin)
|   |-- prompts/            # Prompt templates
|   |-- models/             # Data models
|-- data/
|   |-- synthetic/          # Generated test data (5 JSON files)
|   |-- documents/          # University policy documents (7 files)
|   |-- vector_store/       # ChromaDB storage
|-- docs/                   # All documentation
|-- mock_erp/               # Mock ERP server for testing
|-- scripts/                # Utility scripts
|-- tests/                  # Automated tests
|-- requirements.txt        # Python dependencies
|-- .env.example            # Config template
|-- README.md               # Repository README
```

10. GitHub Repository

https://github.com/zotac-pc/technify-ai-assistant

Git Workflow Rules

- NEVER push directly to 'main' branch
- Create feature branches: feature/jwt-auth, feature/mock-erp, etc.
- Commit frequently with clear messages
- Create Pull Requests for code review
- Team Lead reviews and merges PRs

11. Deliverables & Success Criteria

#	Deliverable	Description	Deadline
1	Architecture Diagram	System diagrams & components	Week 1 (DONE)
2	Prompt Library	All prompt templates	Week 2
3	Knowledge Base	Docs in searchable format	Week 3
4	Working Prototype	Web-based chatbot	Week 5
5	ERP Integration	API connectors	Week 5
6	Documentation	Install, API, User guides	Week 7

Success Criteria

Criteria	Target	How to Test
Response Time	< 3 seconds	Measure with timer
Accuracy	90%+	Test 100 questions
Data Leakage	0%	Try accessing others' data
ERP Integration	Successful	API calls return correct data
Role-Based Security	Implemented	Each role sees only their data

12. Team Roles & Assignments

#	Role	Responsibilities
1	Team Lead + AI Architect	Architecture, LangChain pipeline, code review, coordination
2	Backend Developer 1	FastAPI endpoints, JWT auth, API gateway, WebSocket
3	Backend Developer 2	ERP API connectors, data formatting, audit logging, mock ERP
4	AI/NLP Developer	LangChain chains, prompt engineering, conversation mgmt
5	KB / RAG Developer	ChromaDB setup, document ingestion, RAG pipeline, chat UI
6	Data Engineer + QA	Synthetic data, testing, documentation, quality assurance

13. 8-Week Timeline

Week	Phase	Key Tasks
Week 1	Setup & Architecture	Git, venv, architecture diagram, research
Week 2	Foundation	JWT auth, mock ERP, basic LangChain, prompts
Week 3	Core Features	Student features, ERP connectors, RAG, memory
Week 4	Advanced Features	Faculty/admin features, study planner
Week 5	Integration & UI	Chat UI, WebSocket, full integration
Week 6	Security & Testing	Penetration testing, accuracy, performance
Week 7	Docs & Polish	Documentation, open-source LLM, cleanup
Week 8	Final Demo	Integration with real ERP, demo to CEO

14. Week 1 Tasks - Detailed (June 7-13)

ALL MEMBERS - Day 1 (June 7)

- Read the complete PROJECT_DOCUMENT.md in the docs/ folder
- Install Python 3.10+, Git, VS Code
- Clone the GitHub repo: git clone https://github.com/zotac-pc/technify-ai-assistant
- Create virtual environment: python -m venv venv
- Activate venv: venv\Scripts\activate (Windows)
- Install dependencies: pip install -r requirements.txt
- Copy .env.example to .env
- Run server: uvicorn app.main:app --reload
- Visit http://localhost:8000/docs to verify Swagger UI loads
- Run data generator: python scripts/generate_data.py

Member A - Backend Dev 1 (Auth)

- Learn FastAPI (YouTube tutorial + docs)
- Study JWT tokens (jwt.io)
- Learn python-jose library
- Design auth middleware (pseudocode)
- Build app/auth/jwt_handler.py
- Build app/auth/rbac.py
- DELIVERABLE: JWT validation working by June 13

Member B - Backend Dev 2 (ERP Connector)

- Learn httpx library (async HTTP client)
- Study synthetic data JSON structure
- Design mock ERP API endpoints
- Build mock_erp/main.py (FastAPI on port 8001)
- Build 5+ mock endpoints serving test data
- DELIVERABLE: Mock ERP running by June 13

Member C - AI/NLP Developer

- Learn LangChain (tutorial + docs)
- Practice OpenAI API calls
- Build simple chatbot with ConversationBufferMemory
- Design prompt templates for attendance, results, courses
- DELIVERABLE: Basic LangChain chatbot by June 13

Member D - Knowledge Base / RAG Developer

- Learn RAG concepts (YouTube + docs)
- Learn ChromaDB (tutorial + getting started)
- Review 7 policy documents in data/documents/
- Build document ingestion script (split, embed, store)
- Test: Query 'attendance policy' returns relevant chunks
- DELIVERABLE: ChromaDB loaded with docs by June 13

Member E - Data Engineer + QA

- Validate synthetic data (run generate_data.py)
- Check data quality (unique IDs, correct links)
- Add timetable and assignment data to generator
- Setup pytest framework + conftest.py
- Write 5 basic tests (health endpoint, data files, etc.)
- Create docs/data_dictionary.md
- DELIVERABLE: Enhanced data + test framework by June 13

Team Lead Tasks

- Initialize GitHub repo + push code (DONE)
- Create architecture diagram (DONE - docs/architecture.md)
- Design API endpoints (DONE - docs/api_guide.md)
- Coordinate with Web/Backend team leads
- Set up daily standup meetings
- Weekly review meeting on Friday June 13

15. Quick Setup Guide

```
# Clone the repository
git clone https://github.com/zotac-pc/technify-ai-assistant
cd technify-ai-assistant

# Create and activate virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
source venv/bin/activate        # Mac/Linux

# Install dependencies
pip install -r requirements.txt

# Setup environment config
copy .env.example .env
# Edit .env with your API keys

# Generate test data
python scripts/generate_data.py

# Run the server
uvicorn app.main:app --reload
```

```
# Open in browser
# http://localhost:8000/docs
```

Let's Build Something Amazing!

AI Team 1 - Technify Software House

June - July 2026

GitHub: https://github.com/zotac-pc/technify-ai-assistant

This document is confidential.

For AI Team 1 members only.